

torch.hann_window#

https://pytorch.org/docs/stable/generated/torch.hann_window.html

Description:

Hann window function. where NNN is the full window size. The input `window_length` is a positive integer controlling the returned window size. `periodic` flag determines whether the returned window trims off the last duplicate value from the symmetric window and is ready to be used as a periodic window with functions like `torch.stft()`. Therefore, if `periodic` is true, the NNN in above formula is in fact $\text{window_length} + 1$. Also, we always have `torch.hann_window(L, periodic=True)` equal to `torch.hann_window(L + 1, periodic=False)[: -1]`. If `window_length = 1`, the returned window contains a single value 1.

Function Signature

```
torch.hann_window(window_length, periodic=True, *,  
dtype=None, layout=torch.strided, device=None,  
requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Returns

Return type

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported. `layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). `device` will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `requires_grad` (`bool`, optional) – If autograd should record operations on the returned tensor. Default: `False`.

Returns:

Tensor

Notes & Warnings

NOTE

Note If `window_length = 1`, the returned window contains a single value 1.

Detailed Documentation

Documentation

Hann window function.

where NNN is the full window size.

The input `window_length` is a positive integer controlling the returned window size. `periodic` flag determines whether the returned window trims off the last duplicate value from the symmetric window and is ready to be used as a periodic window with functions like `torch.stft()`. Therefore, if `periodic` is `true`, the NNN in above formula is in fact $\text{window_length} + 1 - \text{window_length} + 1 = \text{window_length} + 1$. Also, we always have `torch.hann_window(L, periodic=True)` equal to `torch.hann_window(L + 1, periodic=False)[-1])`.

If `window_length = 1`, the returned window contains a single value 1.

`window_length` (int) – the size of returned window

`periodic` (bool, optional) – If True, returns a window to be used as periodic function. If False, return a symmetric window.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported.

`layout` (torch.layout, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

A 1-D tensor of size $(\text{window_length},)$ containing the window